

Report - Version 6: Tool Registry

Introduction

Version 6 introduces the Tool Registry.

In the previous version, the project introduced the Memory Manager. The agent was already able to select a tool, create an execution plan, extract the required arguments, execute the selected tool, and manage the interaction history through a dedicated memory component.

Version 6 keeps this structure, but changes how tools are organized. Tool functions, parameters, descriptions, and synonyms are now stored together in a central registry.

This version is still rule-based. No LLM is used, no database is introduced, and no external framework is introduced.

Goal of Version 6

The main goal of Version 6 is to introduce centralized tool management.

The agent components should no longer depend on separate structures for tool definitions and routing synonyms. They should retrieve the information they need from a shared registry.

Each registered tool contains:

- the tool function;
- the required parameters;
- a description of what the tool does;
- the synonyms used by the rule-based Router.

The important idea is simple: each tool should be registered once and its information should be available from one central place.

What changes compared to Version 5

Version 5 focused on the Memory Manager. Memory operations were moved away from the Agent and delegated to dedicated functions.

Version 6 does not modify the Memory Manager as the main concept. Memory continues to be managed in the same way.

The new component is the Tool Registry.

In Version 5, tool metadata was stored in the tools dictionary, while routing synonyms were stored separately in tool_synonyms.

In Version 6, these structures are unified into tool_registry. The Router, Planner, Parser, and Executor now read the information they need from the same registry.

New concept: Tool Registry

The Tool Registry is a central structure that stores the tools available to the agent together with their metadata.

The registry starts as an empty dictionary:

```
tool_registry = {}
```

Tools are added through a dedicated registration function:

```
def register_tool(name, function, parameters, description, synonyms):
    tool_registry[name] = {
        "function": function,
        "parameters": parameters,
        "description": description,
        "synonyms": synonyms
    }
```

This keeps registration explicit and easy to inspect while avoiding separate definitions for the same tool.

Tool registration

Each tool is registered with the information required by the other agent components.

For example, the addition tool is registered with its function, two parameters, a short description, and the synonyms used for routing:

```
register_tool(
    name="addition",
    function=addition,
    parameters=["a", "b"],
    description="Add two numbers.",
    synonyms=["add", "addition", "sum", "plus", "+"]
)
```

The same pattern is used for greeting, subtraction, multiplication, division, and power.

How the components use the Tool Registry

The Tool Registry is not an extra execution step. It is a shared source of information used by several components.

The Router reads the synonyms of each registered tool to score the user request.

The Planner reads the description and required parameters of the selected tool.

The Parser reads the parameters required by the selected tool and extracts the corresponding arguments.

The Executor reads the registered function and executes it after validating the parsed arguments.

This reduces duplicated tool information and makes the responsibilities of the components easier to understand.

Complete architecture in Version 6

The execution flow of Version 6 remains:

Request → Router → Planner → Generic Parser → Executor → Memory Manager → Memory

↓

Output

The Tool Registry supports the Router, Planner, Parser, and Executor by providing the definitions and metadata of the available tools.

The main responsibilities are now clearer:

Tool Registry = stores tools and their metadata

Router = selects the tool using registered synonyms

Planner = creates the execution plan using registered metadata

Parser = extracts the arguments required by the selected tool

Executor = validates and executes the registered function

Memory Manager = saves, reads, and clears memory

Agent = coordinates

Memory = stores the state

The Agent remains the coordinator. The main architectural change is that tool information now has one central source.

Agent state in Version 6

The agent state keeps the same structure introduced in Version 4 and used in Version 5.

A typical state looks like this:

```
{
  "request": "What is 2 + 3?",
  "plan": {
    "tool": "addition",
    "reason": "Add two numbers.",
    "needs_arguments": ["a", "b"]
  },
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}
```

The state itself does not need a new key for the Tool Registry. The registry is an architectural component used to obtain tool information before and during execution.

Unsupported requests

Unsupported requests continue to be handled correctly.

For example, the request:

```
agent("What is the weather in Rome?")
```

does not match the synonyms of any registered tool.

The Router therefore returns no tool, the Planner creates a plan with no selected action, and the Agent returns a safe fallback result.

The interaction is still saved through the Memory Manager, exactly as in Version 5.

Tests

The tests verify that the agent continues to work after introducing the Tool Registry.

The main test cases are:

```
clear_memory()
tool_registry
```

```
agent("What is 2 + 3?")
agent("Hello, what is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
get_memory()
get_last_interaction()
```

These tests verify that:

- the Tool Registry contains every available tool together with its function, parameters, description, and synonyms;
- the Router still prioritizes the main intent;
- the Planner still creates a plan before execution;
- the Parser still extracts the required arguments;
- the Executor retrieves and executes the function stored in the registry;
- unsupported requests are handled without selecting a tool;
- the Memory Manager continues to save, read, and clear memory;
- memory continues to store the full state of each interaction.

What Version 6 teaches

Version 6 teaches an important agent design principle:

tool management should be centralized

The Tool Registry is still simple, but it introduces a single source of truth for the tools available to the agent.

This makes the architecture easier to extend because a new tool can be registered together with the information required by the Router, Planner, Parser, and Executor.

It also prepares the project for the next evolution, where tool descriptions can become useful to a language-model-based Router.

Limitations

Version 6 improves the architecture, but some limitations remain:

- the Router is still rule-based and depends on manually defined synonyms;
- the Planner is still rule-based;
- the Parser still uses manual extraction rules;
- memory is still a simple list;
- the Tool Registry is still a simple in-memory Python dictionary;
- tool registration is still manual;
- there is no LLM yet;
- the agent still performs only one action at a time.

These limitations are expected and will be addressed gradually in the next versions.

Conclusion

Version 6 is a natural evolution of Version 5.

The Memory Manager remains unchanged, but the new central element is the Tool Registry.

Tool functions, parameters, descriptions, and synonyms are now registered together. The Router, Planner, Parser, and Executor use this shared information instead of relying on separate tool structures.

This makes tool management cleaner, more organized, and easier to extend without changing the educational simplicity of the project.

The next step will be Version 7, where the project will introduce an LLM Router.